

Neural Option Pricing: A Deep Learning Approach for Option Pricing

Emanuele Saccoliti

The BS Model

Recall the Black-Scholes formula for a European call:

$$C(\tau, S) = S \Phi(d_1(\tau, S)) - Ke^{-r\tau} \Phi(d_2(\tau, S)) \quad (1)$$

with $C : \mathbb{R}_+ \times (0, T] \rightarrow \mathbb{R}$, where $S = S_t$ is the price of the underlying stock, $\tau = T - t$ is time to expiry and

$$\Phi(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^x e^{-\frac{z^2}{2}} dz, \quad \forall x \in \mathbb{R}$$

is the cumulative distribution function of a standardized normal random variable, and

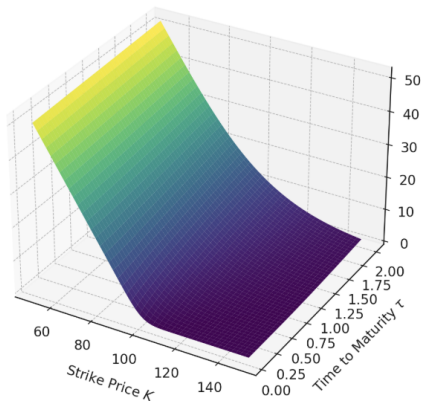
$$d_1(\tau, S) = \frac{\ln \frac{S}{K} + (r + \frac{1}{2}\sigma^2)\tau}{\sigma\sqrt{\tau}},$$

and

$$d_2(\tau, S) = d_1(\tau, S) - \sigma\sqrt{\tau}.$$

The option price is a function of five variables, the asset price S , the strike price K , the time to maturity τ , the volatility σ and the risk free interest rate r .

As the option price is linear homogenous in S and K , it can be reduced to four variables, fixing $S = 100$ for example; that is the price of a call for example can be expressed as $C(100, K, \tau, \sigma, r)$.



The Pricing Map

Let $\mathcal{M} := \mathcal{M}(\theta)$ be a general abstract model, where θ is the vector of parameters in the set $\Theta \subset \mathbb{R}^n$, for some $n \in \mathbb{N}$. Thus the model $\mathcal{M}(\theta)$ and the corresponding prices of financial contracts are fully specified by the choice of the parameter combination $\theta \in \Theta$.

In the Black-Scholes case, we have $\mathcal{M}^{BS}(\theta)$ with the vector of parameters $\theta := (K, \tau, r, \sigma) \in \Theta \subset \mathbb{R}^4$.

Next, let φ be the family of financial products we aim to price for a set of strikes K and maturities T .

In the Black-Scholes case, the pricing functional can be described by the pricing map

$$P(\mathcal{M}^{BS}(\cdot)) : \Theta \rightarrow P(\mathcal{M}^{BS}), \quad \theta \mapsto P(\mathcal{M}^{BS}(\theta), \varphi),$$

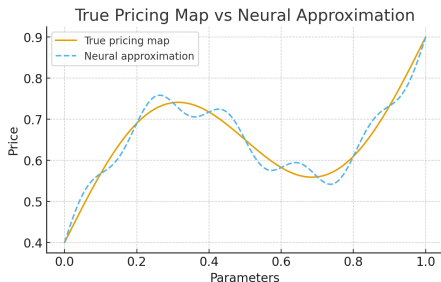
where $P(\mathcal{M}^{BS}(\theta), \varphi) = \mathbb{E}^{\mathbb{Q}_\theta} [\varphi(X(\theta))]$, for some underlying stochastic process X^θ .

Functional Approximation

Our goal is to construct an feed-forward Neural Network to replace the pricing map; that is, to approximate the pricing map for the set of model parameters Θ

$$\hat{P}(\mathcal{M}(\cdot), \varphi) : \Theta \rightarrow \mathbb{R}, \quad \theta \mapsto \hat{P}(\mathcal{M}(\theta), \varphi) \approx P(\mathcal{M}(\theta), \varphi).$$

A feed-forward network defines a parametric mapping $y = F(x, z)$ and the training calibrates the optimal values of network parameters $\hat{z}$ that result in the best function approximation $F^*(\cdot) \approx F(\cdot, z)$ of the unknown function $F^*(\cdot)$ for the pairs of input and output data (x, y) .



ANNs

Let $L \in \mathbb{N}$ be the number of layers (depth), and let $(d_0, d_1, \dots, d_L) \in \mathbb{N}^{L+1}$ denote the number of (neurons) on each layer, where d_0 and d_L correspond to the input and output layers, respectively.

For each layer $\ell = 1, \dots, L$, define the affine transformation

$$\mathcal{T}_\ell : \mathbb{R}^{d_{\ell-1}} \longrightarrow \mathbb{R}^{d_\ell}, \quad \mathcal{T}_\ell(x) = W_\ell x + b_\ell, \quad (2)$$

where $W_\ell \in \mathbb{R}^{d_\ell \times d_{\ell-1}}$ is the weight matrix, $b_\ell \in \mathbb{R}^{d_\ell}$ is the bias vector, and $x \in \mathbb{R}^{d_{\ell-1}}$ denotes the input vector to layer ℓ (i.e., the vector of activations from the previous layer); that is

$$x = (h_1^{(\ell-1)}, h_2^{(\ell-1)}, \dots, h_{d_{\ell-1}}^{(\ell-1)}).$$

The product $W_\ell \otimes x$ is a vector $\in \mathbb{R}^{d_\ell}$.

Each entry $(W_\ell)_{ij}$ represents the weight connecting node $j \in d_{\ell-1}$ of layer $\ell - 1$ with node $i \in d_\ell$ of layer ℓ .

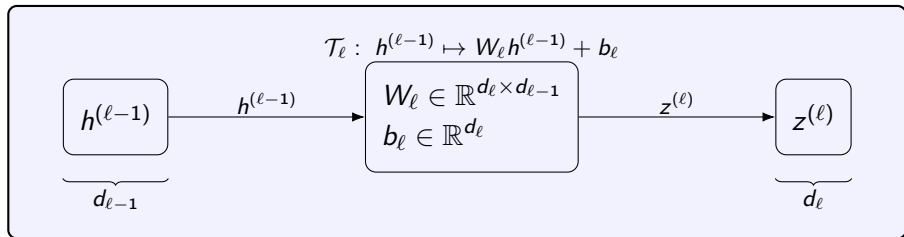
For each layer, the corresponding *pre-activation* vector is given by

$$z^{(\ell)} := \mathcal{T}_\ell(h^{(\ell-1)}) = W_\ell h^{(\ell-1)} + b_\ell, \quad \ell = 1, \dots, L,$$

where $h^{(0)} := x$ denotes the input vector.

Component-wise, this reads

$$z_i^{(\ell)} = \sum_{j=1}^{d_{\ell-1}} (W_\ell)_{ij} h_j^{(\ell-1)} + b_i^{(\ell)}. \quad (3)$$



Example 1

Suppose $d_{\ell-1} = 3$ e $d_{\ell} = 2$. Thus we have

- W_{ℓ} is a 2×3 weight matrix, b_{ℓ} is a 2×1 bias vector, x is a 3×1 vector.
- $x = (x_1, x_2, x_3)^{\top}$:

$$z_1^{(\ell)} = (W_{\ell})_{11}x_1 + (W_{\ell})_{12}x_2 + (W_{\ell})_{13}x_3 + b_1^{(\ell)},$$

$$z_2^{(\ell)} = (W_{\ell})_{21}x_1 + (W_{\ell})_{22}x_2 + (W_{\ell})_{23}x_3 + b_2^{(\ell)}.$$

The corresponding *activation* at layer ℓ is then defined as

$$h^{(\ell)} := \phi_{\ell}(z^{(\ell)}) = \phi_{\ell}(W_{\ell}h^{(\ell-1)} + b_{\ell}) \quad (4)$$

where $\phi_{\ell} : \mathbb{R} \rightarrow \mathbb{R}$ is referred to as the activation function. It is typically nonlinear (such as ReLU, tanh, or sigmoid etc...) and applied component-wise on the outputs of the affine map $\mathcal{T}_{\ell}$.

Example 2

Consider the previous example and suppose $x = (x_1, x_2, x_3) = (1, 0.5, -2)$,

$$W_\ell = \begin{pmatrix} 1.2 & -0.3 & 0.7 \\ 0.5 & 0.8 & -0.1 \end{pmatrix}, \quad b_\ell = \begin{pmatrix} 0.2 \\ -0.5 \end{pmatrix}.$$

Thus we have:

$$z_1^{(\ell)} = 1.2(1) - 0.3(0.5) + 0.7(-2) + 0.2 = -0.9,$$

$$z_2^{(\ell)} = 0.5(1) + 0.8(0.5) - 0.1(-2) - 0.5 = 0.8.$$

Suppose $\phi(x) = \text{ReLU}(x) = \max(0, x)$, we then have:

$$h_1^{(\ell)} = \max(0, -0.9) = 0,$$

$$h_2^{(\ell)} = \max(0, 0.8) = 0.8.$$

Therefore the activation vector at layer ℓ is $h^{(\ell)} = \begin{pmatrix} 0 \\ 0.8 \end{pmatrix}$.

We denote as $\mathcal{T} = (\mathcal{T}_1, \dots, \mathcal{T}_L)$ the collection of all the affine functions of the form (2) on each layer. Then, a Neural Network $F(\mathcal{T}, \cdot) : \mathbb{R}^{d_0} \rightarrow \mathbb{R}^{d_L}$ is defined as a chain of affine and activations functions

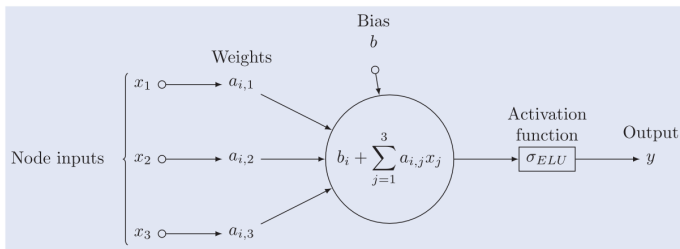
$$F(x) := (\phi_L \circ \mathcal{T}_L) \circ \dots \circ (\phi_1 \circ \mathcal{T}_1)(x), \quad (5)$$

so that $F : \mathbb{R}^{d_0} \rightarrow \mathbb{R}^{d_L}$ maps input features to the network output.

- The input layer is represented by $h^{(0)} := x$.
- Each subsequent layer applies an affine map followed by the corresponding activation:

$$h^{(\ell)} = (\phi_\ell \circ \mathcal{T}_\ell)(h^{(\ell-1)}), \quad \ell = 1, \dots, L.$$

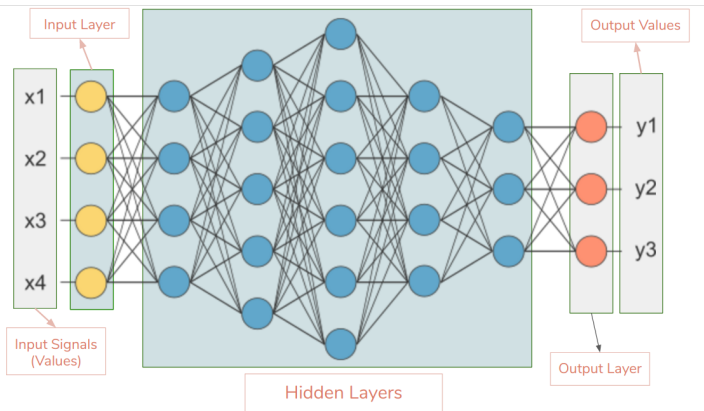
- The intermediate layers $(\phi_\ell \circ \mathcal{T}_\ell)$, for $\ell = 1, \dots, L - 1$, are referred to as *hidden layers*.
- The last layer $(\phi_L \circ \mathcal{T}_L)$ produces the output.



The figure shows the computation performed by a single neuron. The inputs x_1, x_2, x_3 are combined linearly using the weights $a_{i,1}, a_{i,2}, a_{i,3}$ and a bias term b_i , yielding the pre-activation value

$$z_i = b_i + \sum_{j=1}^3 a_{i,j}x_j.$$

This value is then passed through a nonlinear activation function σ (here the ELU function) to produce the neuron's output $y = \sigma(z_i)$. This mapping is applied at every neuron throughout the network, enabling nonlinear function approximation.



The image displays a simple feedforward NN architecture mapping $h^{(0)} = x$ to $h^{(L)} = y$. Each layer consists of an affine transformation followed by an activation

$$h^{(\ell)} = (\phi_{\ell} \circ \mathcal{T}_{\ell})(h^{(\ell-1)}), \quad \ell = 1, \dots, L.$$

Universal Approximation Theorem

A key theoretical cornerstone in Deep Learning is the Universal Approximation Theorem, which establishes that sufficiently expressive neural architectures can approximate any continuous function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ on a compact domain, to arbitrary accuracy.

Theorem

Let $\mathcal{NN}_{d_0, d_1}^\delta$ be the set of neural networks with activation function $\delta : \mathbb{R} \rightarrow \mathbb{R}$, input dimension $d_0 \in \mathbb{N}$ and output dimension $d_1 \in \mathbb{N}$. Then, if δ is continuous and non-constant, $\mathcal{NN}_{d_0, 1}^\delta$ is dense in $L^p(\mu)$ for all finite measures μ .

In other words, for every function $f \in L^p(\mu)$ and for every $\varepsilon > 0$, there exists a neural network $z \in \mathcal{NN}_{d_0, 1}^\delta$ such that

$$\|f - z\|_{L^p(\mu)} < \varepsilon.$$

In a nutshell, we are saying that a one-hidden-layer NN can approximate any continuous function on a compact domain, provided it has a sufficient number of neurons.

Learning the Pricing Map

The objective is to train the NN to learn the mapping between the input variables (r, K, τ, σ) and the corresponding Black–Scholes price.

- A grid of values is generated for the four input variables (r, K, τ, σ) , which will serve as the training inputs for the network.
- The underlying asset price S is fixed at 100 for all options in the dataset, exploiting the homogeneity property of the Black–Scholes formula.
- For each combination (r, K, τ, σ) , the theoretical Black-Scholes price is computed.
- The resulting dataset is stored as a NumPy array, where the first four columns contain the input features $X = (r, K, \tau, \sigma)$ and the fifth column contains the corresponding option price $y = C_{BS}(S, K, \tau, r, \sigma)$.

- The dataset is then divided in:
 - ▶ $X_{\text{train}}, y_{\text{train}}$ (75%) $\rightarrow$ used for the NN training,
 - ▶ $X_{\text{test}}, y_{\text{test}}$ (25%) $\rightarrow$ kept separate to evaluate the model's performance on unseen data.
- We construct and train the NN to approximate the pricing map $\theta \rightarrow P(\mathcal{M}^{BS}(\theta), \varphi)$, providing the approximation $\hat{P}(\cdot) \approx P(\cdot)$

Remark

We parameterize the neural network by ϑ , and denote its output by $\hat{P}_{\vartheta}(\theta)$. The parameters ϑ are learned by minimizing the mean squared error between the true prices and the network predictions over the training set:

$$\vartheta^* \in \arg \min_{\vartheta} \frac{1}{N} \sum_{i=1}^N (\hat{P}_{\vartheta}(\theta_i) - P(\theta_i))^2, \quad \theta_i \sim \Theta_{\text{train}}.$$

In other words, the network learns the pricing function $P(\cdot)$ by adjusting its internal weights ϑ so that $\hat{P}_{\vartheta}(\cdot) \approx P(\cdot)$ over the training domain. Once trained, the network serves as a fast surrogate model for option pricing.

Homogeneity Test

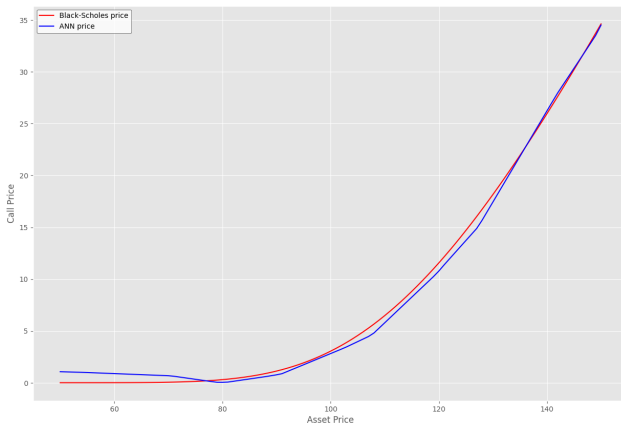
The goal is to verify whether the NN correctly reproduces the homogeneity of degree one property of option prices with respect to S and K , that is

$$C(\lambda S, \lambda K, T, r, \sigma) = \lambda C(S, K, T, r, \sigma).$$

This tests whether the NN has learned the structural relationship between S and K , rather than merely fitting local data patterns.

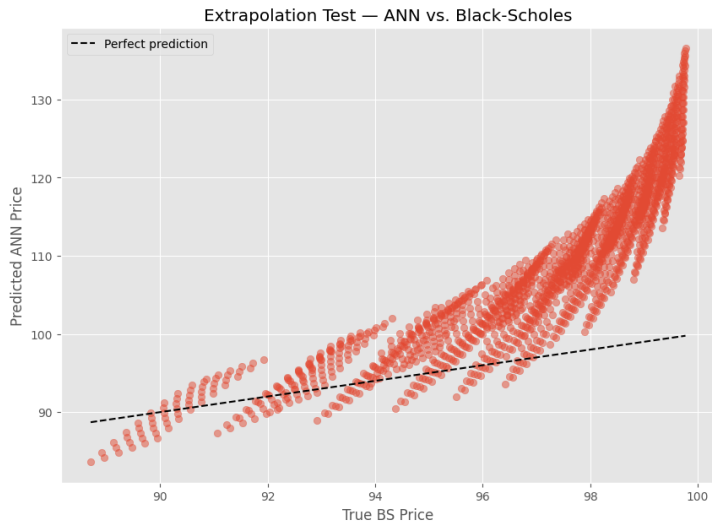
In particular, scaling both the underlying price and the strike should result in a proportional scaling of the option value, independently of the maturity and volatility parameters.

By checking this condition, we can assess whether the NN has internalized the intrinsic structure of the pricing function.



- On the left side, the call option price computed by our NN increases when the asset price decreases which violates the call spread arbitrage condition.
- Indeed, nothing guarantees that the no arbitrage conditions such as the call spread, butterfly spread or calendar spread are not violated when we build the NN. Therefore, it may be required additional regularisation techniques or constrains.

Extrapolation



- The plot compares the NN-predicted option prices against the true Black–Scholes values for previously unseen regions of the input space.
- The network's performance deteriorates when extrapolating beyond the range of inputs on which it was trained.
- The deviation from the 45-degree “perfect-prediction” line demonstrates that outside the training support the network doesn't maintain the correct price structure.
- In particular, it tends to overestimate option values in high-price regions and shows unstable behaviour at the tails.

References

- Blanka Horvath, Aitor Muguruza, Mehdi Tomas (2021): "Deep learning volatility"
- Daniel de Souza Santos, Tiago Ferreira (2024): "Neural Network Learning of Black-Scholes Equation for Option Pricing"
- Ian Goodfellow, Yoshua Bengio, Aaron Courville (2016): "Deep Learning"
- Johannes Ruf, Weiguan Wang (2020): "Neural networks for option pricing and hedging: a literature review".
- Kurt Hornik (1991): "Approximation capabilities of multilayer feedforward networks"
- Kurt Hornik, Maxwell Stinchcombe, Halbert White (1989): "Multilayer feedforward networks are universal approximators"
- Kurt Hornik, Maxwell Stinchcombe, Halbert White (1990): "Universal approximation of an unknown mapping and its derivatives using multilayer feedforward networks"
- Yang Qu, Ming-Xi Wang (2019): "The option pricing model based on time values: an application of the universal approximation theory on unbounded domains"